

Classical Statistics for Scientific Data Analysis

Monte Carlo methods

Pueh Leng Tan, plt2120@columbia.edu
14 February 2026

Random number generators

Starts with random seed, for some fixed a and m

$$n_{i+1} = (an_i) \bmod m \in [1, m-1]$$


$$r_i = \frac{n_i}{m} \in (0, 1)$$

(And then some magic to add it 0 and 1)

Random numbers → Uniform distribution → Any distribution

Probability Integral Transform

$$X \sim f(x)$$

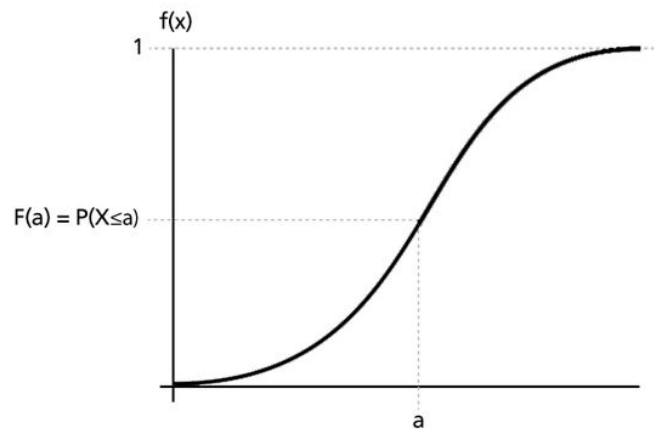
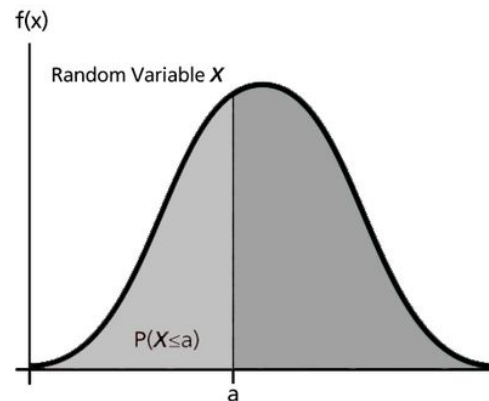
PDF

$$F_X(x) \equiv \int_{-\infty}^x f(\tilde{x}) d\tilde{x}$$

CDF

$$Y \equiv F_X(x)$$

$$Y \sim \mathcal{U}_{[0,1]}$$



CDF is r.v. with standard uniform distribution

Probability Integral Transform

$$\begin{aligned}F_Y(y) &= \mathbf{P}(Y \leq y) \\&= \mathbf{P}(F_X(X) \leq y) \\&= \mathbf{P}(X \leq F_X^{-1}(y)) \\&= F_X(F_X^{-1}(y)) \\&= y\end{aligned}$$

Definition of CDF of Y

Definition of Y

Apply inverse CDF on both sides

Definition of CDF of X

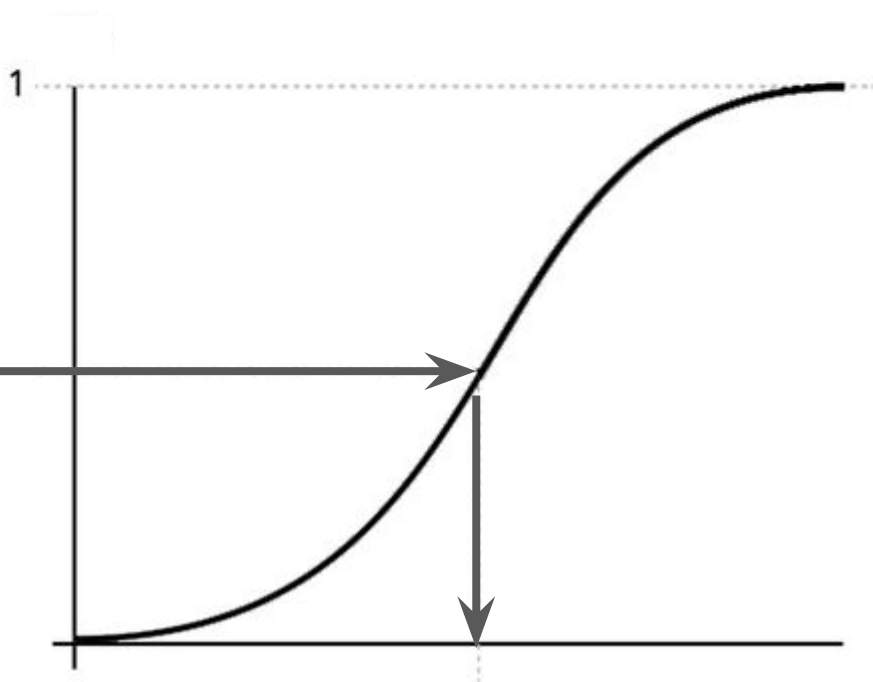
CDF of y = CDF of standard uniform
Hence $y \sim$ standard uniform

Inverse transform sampling

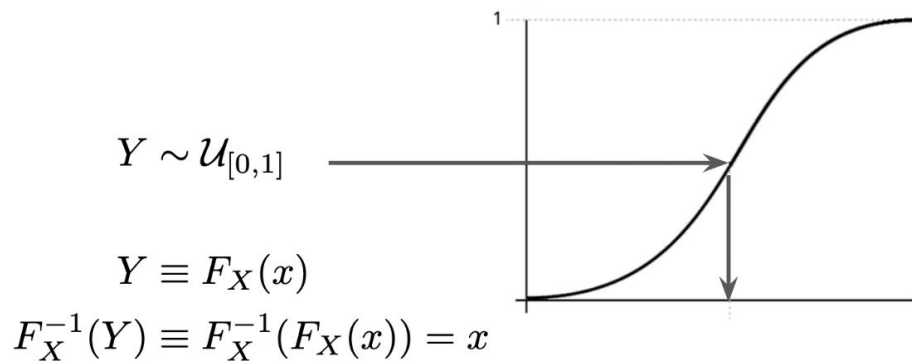
$$Y \sim \mathcal{U}_{[0,1]}$$

$$Y \equiv F_X(x)$$

$$F_X^{-1}(Y) \equiv F_X^{-1}(F_X(x)) = x$$



Inverse transform sampling



Use the inverse transform sampling to sample from Gaussian($\mu=1.1$, $\sigma=0.3$)

Coding exercise after
break

Back 10:50 am

Coding exercise

Inverse transform sampling (15 mins)

Clues:

Inverse CDF = Percent point function

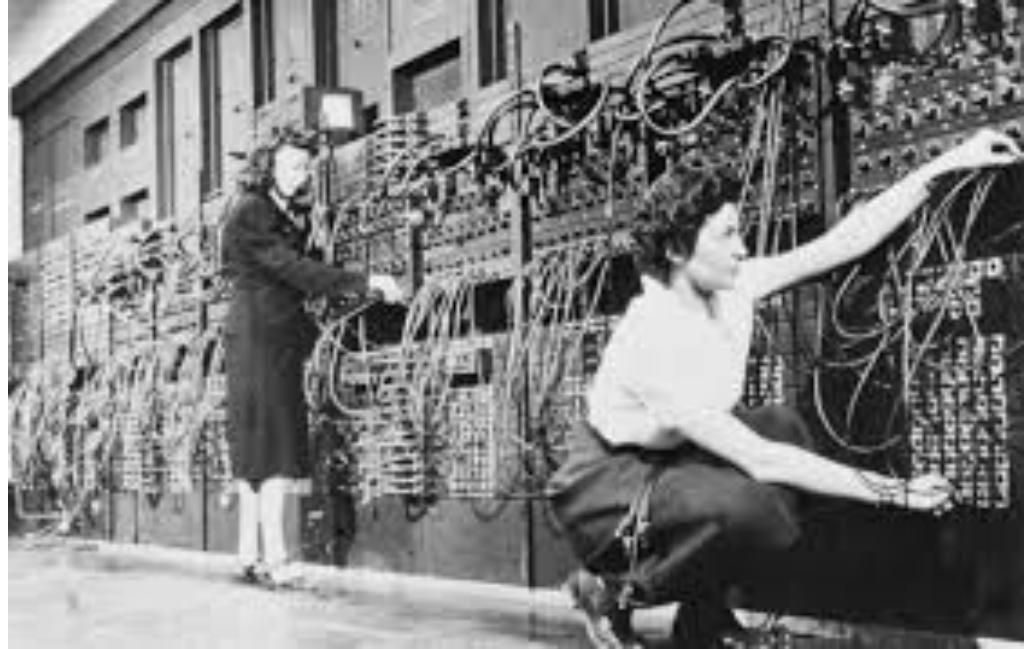
Inverse CDF = `sps.norm.ppf(loc=mu, scale=sigma)`

Break (15 mins)

Coding exercise after break
Back 10:50 am

Monte Carlo methods

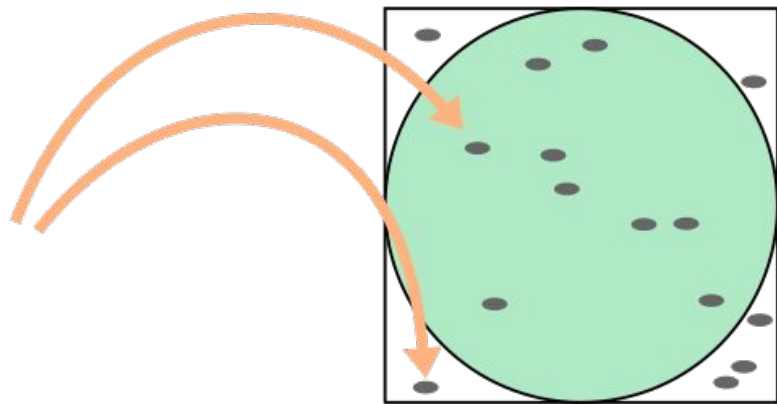
- Broad class of computational algorithm
 - Solve problems numerically via repeated random sampling
- Stanisław Ulam, late 1940s
 - Monte Carlo Casino in Monaco
 - ENIC to simulate probability of winning solitaire
 - H-bomb
- Uses
 - Generating samples from distribution
 - Numerical integration
 - Optimization



Electronic Numerical Integrator and Computer (ENIC)

Estimating π

$$\begin{aligned}\frac{\text{hits}}{\text{trials}} &\approx \frac{\text{Area of circle}}{\text{Area of square}} \\ &= \frac{\pi r^2}{4r^2} \\ &= \frac{\pi}{4}\end{aligned}$$

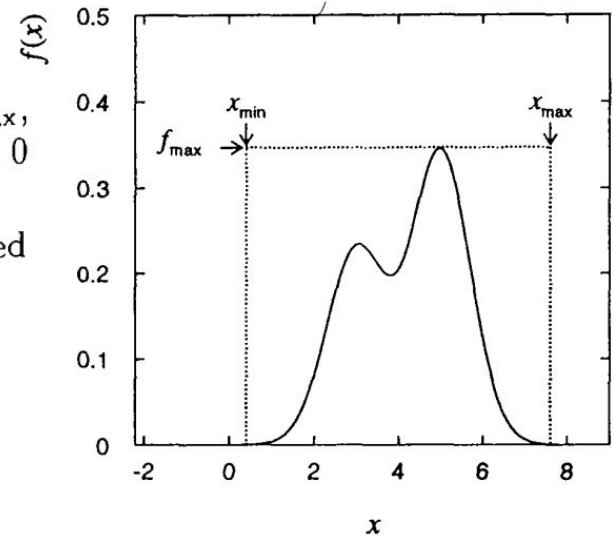


Coding exercise

Estimate Pi (15 mins)

Accept-Reject algorithm (Rejection sampling)

- (1) Generate a random number x , uniformly distributed between $x_{\min}$ and $x_{\max}$, i.e. $x = x_{\min} + r_1(x_{\max} - x_{\min})$ where r_1 is uniformly distributed between 0 and 1.
- (2) Generate a second independent random number u uniformly distributed between 0 and $f_{\max}$, i.e. $u = r_2 f_{\max}$.
- (3) If $u < f(x)$, then accept x . If not, reject x and repeat.



MCMC
Back 11:50 am

Break (15 mins)

MCMC
Back 11:50 am

Coding exercise

Rejection sampling (20 mins)

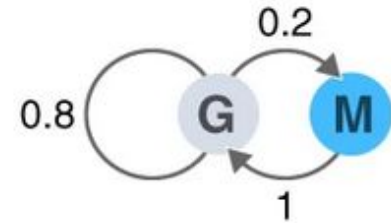
Discrete time Markov Chain

$$\begin{aligned} P(X_{n+1} = x | X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) \\ = P(X_{n+1} = x | X_n = x_n) \end{aligned}$$

Sum of outward going arrows = 1

But sum of inward going arrows not necessarily = 1

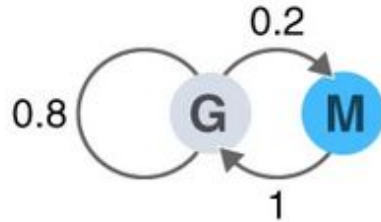
Markov state diagram



Nat Methods **16**, 663–664 (2019)

Transition matrix and equilibrium distribution

Directed graph



Probability of $G \rightarrow M$

Transition matrix

$$\mathbf{T} = \begin{bmatrix} p_{GG} & p_{GM} \\ p_{MG} & p_{MM} \end{bmatrix} = \begin{bmatrix} 0.8 & 0.2 \\ 1 & 0 \end{bmatrix}$$

After 2 steps

$$\mathbf{T}^2 = \begin{bmatrix} 0.84 & 0.16 \\ 0.80 & 0.20 \end{bmatrix}$$

After 4 steps

$$\mathbf{T}^4 = \begin{bmatrix} 0.83 & 0.17 \\ 0.83 & 0.17 \end{bmatrix} \approx \lim_{n \rightarrow \infty} \mathbf{T}^n$$

Equilibrium distribution

Monte Carlo Markov Chains (MCMC)

- Class of sampling algorithm
 - Random walk given some initial position
 - Markov chain's equilibrium distribution = Target distribution
- Advantages
 - More efficient in higher dimensions than other sampling methods
- Disadvantages
 - Potentially slow convergence
 - Autocorrelation
 - Difficult diagnostics
- Random walk MCMC
 - Metropolis
 - Metropolis-Hastings
 - Hamiltonian Monte Carlo
 - etc..

Metropolis Algorithm

Let $f(x)$ be a function that is proportional to the desired probability density function $P(x)$ (a.k.a. a target distribution)^[a].

1. Initialization: Choose an arbitrary point x_t to be the first observation in the sample and choose a proposal function $g(x | y)$. In this section, g is assumed to be symmetric; in other words, it must satisfy $g(x | y) = g(y | x)$.
2. For each iteration t :
 - *Propose* a candidate x' for the next sample by picking from the distribution $g(x' | x_t)$.
 - *Calculate* the *acceptance ratio* $\alpha = f(x')/f(x_t)$, which will be used to decide whether to accept or reject the candidate^[b]. Because f is proportional to the density of P , we have that $\alpha = f(x')/f(x_t) = P(x')/P(x_t)$.
 - *Accept or reject*:
 - Generate a uniform random number $u \in [0, 1]$.
 - If $u \leq \alpha$, then *accept* the candidate by setting $x_{t+1} = x'$,
 - If $u > \alpha$, then *reject* the candidate and set $x_{t+1} = x_t$ instead.

Coding exercise

Metropolis algorithm

Questions?

Thanks!